Lab 5

Name: Mahdi Davoodzadeh
SN: 405725536
Repository: https://github.com/deep-observability-lab/Linux-Kernel-Tracing/tree/main/LABS/LAB1

1. Objective

In this lab, I investigated how the Maximum Transmission Unit (MTU) affects IPv4 packet transmission. I first created a small virtual network with an MTU of **1500 bytes**. I sent a normal ICMP Echo Request and then sent a larger request with **2000 bytes of Ping data**. I compared behavior with fragmentation prohibited and allowed.

I used **LTTng and Trace Compass** to examine the network events and IPv4 header fields, demonstrate that the large packet was divided into fragments, and reconstruct its original length from the recorded data. After satisfying the main assignment, I performed an additional investigation with **tcpdump/Wireshark** to inspect raw frames and reassembly, and with **LTTng system-call tracing and ftrace** to follow the packet from the `ping` program into the kernel's fragmentation and transmission paths.

I analyzed the initial comparison, the coordinated LTTng/PCAP capture, and the ftrace recording as three separate experiments, each with its own packet identities and timestamps.

2. Experimental Environment

2.1 Checking tracing support

I performed the experiment on Ubuntu 24.04.5 LTS with kernel `6.17.0-40-generic`, using LTTng 2.13 and Trace Compass. I used `scripts/01_mtu_preflight.sh` to verify the available tools and network tracepoints. LTTng kernel tracing worked on `6.17.0-40-generic`. The TraceFS definitions for `net_dev_queue`, `net_dev_start_xmit`, `net_dev_xmit`, `netif_receive_skb`, and `skb:kfree_skb` were present. In particular, the network transmit events exposed fields such as `skbaddr`, `len`, and the interface name.

I later confirmed that the LTTng instrumentation for `net_dev_queue` also included parsed IPv4 fields: `tot_len`, `id`, `frag_off`, protocol, and source/destination addresses. These fields proved more useful for the assignment than counting network events alone.

2.2 Setting up an isolated network

I used `scripts/02_setup_mtu_network.sh` to create two Linux network namespaces connected by a virtual Ethernet (`veth`) pair:

Component	Sender	Receiver
Namespace	`lab5_mtu_tx`	`lab5_mtu_rx`
Interface	`l5veth_tx`	`l5veth_rx`
IPv4 address	`10.205.0.1/30`	`10.205.0.2/30`
MTU	**1500 bytes**	**1500 bytes**

This isolated network let me control the link MTU within the two experimental namespaces. The initial connectivity test, with two normal Ping requests, succeeded: two packets were transmitted, two replies were received, and packet loss was 0%.

3. Sending a Packet Larger Than the MTU

I used `scripts/03_test_large_ping.sh` to compare two ways of sending a 2000-byte Ping payload to `10.205.0.2`:

```
# Prohibit IPv4 fragmentation.
sudo ip netns exec lab5_mtu_tx \
  ping -4 -n -c 2 -W 2 -M do -s 2000 10.205.0.2

# Allow IPv4 fragmentation.
sudo ip netns exec lab5_mtu_tx \
  ping -4 -n -c 2 -W 2 -M dont -s 2000 10.205.0.2
```

For an ordinary IPv4 header without options, the original packet size was:

2000 bytes Ping data + 8 bytes ICMP header + 20 bytes IPv4 header = 2028 bytes.

The 2028-byte IPv4 datagram was **528 bytes larger** than the configured 1500-byte MTU.

3.1 Fragmentation prohibited: `-M do`

The first command could not send the oversized IPv4 datagram. It returned exit status **1** and printed:

```
PING 10.205.0.2 (10.205.0.2) 2000(2028) bytes of data.

--- 10.205.0.2 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1013ms

ping: local error: message too long, mtu=1500
ping: local error: message too long, mtu=1500
```

This showed that the local output path enforced the interface MTU when fragmentation was prohibited.

3.2 Fragmentation allowed: `-M dont`

The second command completed with exit status **0** and both large Echo Requests received replies:

```
PING 10.205.0.2 (10.205.0.2) 2000(2028) bytes of data.
2008 bytes from 10.205.0.2: icmp_seq=1 ttl=64 time=0.086 ms
2008 bytes from 10.205.0.2: icmp_seq=2 ttl=64 time=0.118 ms

--- 10.205.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1064ms
rtt min/avg/max/mdev = 0.086/0.102/0.118/0.016 ms
```

The successful Ping established that the large messages could be exchanged when fragmentation was permitted. I then traced transmission to identify the individual fragments and their header fields.

4. LTTng Recording: Normal Ping vs Oversized Ping

I used `scripts/04_lttng_fragmentation_trace.sh` to record two separate LTTng kernel sessions: a **56-byte Ping payload** as the reference and a **2000-byte Ping payload** as the oversized case. Both runs used `-M dont` so that the payload size was the main experimental difference.

For these two initial recordings, LTTng captured `net_dev_queue` and `net_dev_xmit`.

The resulting files were:

```
logs/lttng/04_small_trace_gui/kernel
logs/lttng/04_large_trace_gui/kernel
logs/lttng/04_small_events.txt
logs/lttng/04_large_events.txt
```

Measurement	Normal Ping	Oversized Ping
Ping data (`-s`)	56 B	2000 B
Original IPv4 packet length	84 B	2028 B
Successful requests/replies	2 / 2	2 / 2
System-wide `net_dev_queue` count	26	33
System-wide `net_dev_xmit` count	26	33

The event totals represent system-wide traffic. To identify the experimental fragments, I selected packets on `l5veth_tx` and compared their IPv4 identification, length, and fragment offset fields.

4.1 Normal Ping: one unfragmented IPv4 packet

In Trace Compass, the first `net_dev_queue` event for `l5veth_tx` in the normal-Ping recording showed:

Field	Observed value
Event	`net_dev_queue`
Interface	`l5veth_tx`
`len`	**98 bytes**
IPv4 `tot_len`	**84 bytes**
IPv4 `id`	`0x3C2B`
IPv4 `frag_off`	**0**
Protocol	ICMP
Source → destination	`10.205.0.1 → 10.205.0.2`

The packet fit within the MTU as a single IPv4 datagram. Its sizes were consistent with **56 + 8 + 20 = 84 bytes of IPv4**, or **98 bytes including the 14-byte Ethernet header** visible at this transmit tracepoint.

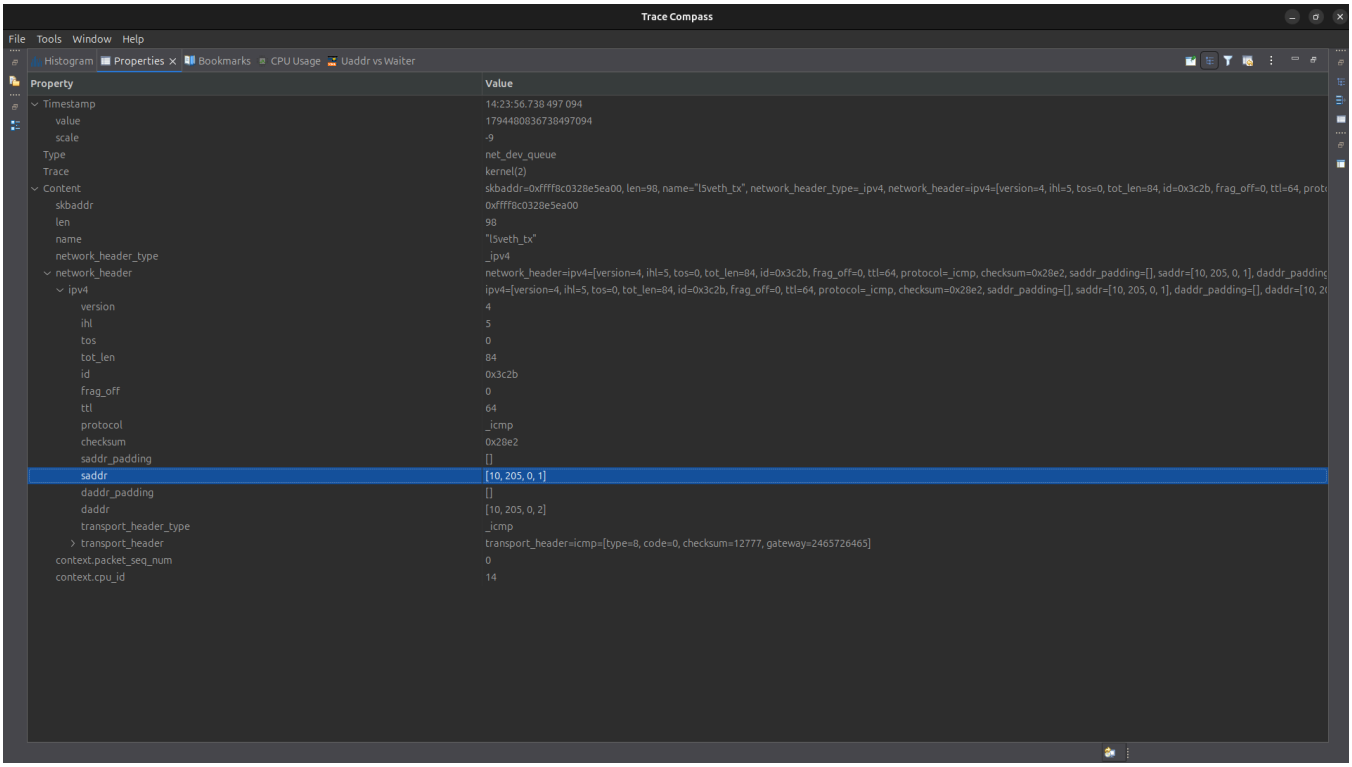


Figure 1 — The normal Echo Request has `tot_len=84` and `frag_off=0`.

4.2 Oversized Ping: two IPv4 fragments

In the large-Ping recording, Trace Compass showed two consecutive `net_dev_queue` events on `l5veth_tx` with the same IPv4 ID, `0x413A`:

Field	Fragment 1	Fragment 2
`len` (with Ethernet header)	**1514 B**	**562 B**
IPv4 `tot_len`	**1500 B**	**548 B**
IPv4 ID	`0x413A`	`0x413A`
Raw IPv4 `frag_off`	8192 (`0x2000`)	185 (`0x00B9`)
More Fragments (MF)	1	0
Fragment data offset	0 B	**1480 B**
Source → destination	`10.205.0.1` → `10.205.0.2`	`10.205.0.1` → `10.205.0.2`

The shared IPv4 ID, source/destination addresses, and protocol linked the two events to a single fragmented datagram. The first fragment had **MF=1** (`0x2000`), meaning that at least one additional fragment followed. The second fragment had MF cleared and an offset of **185 units of eight bytes**, so its data began at byte **185 × 8 = 1480** of the original IPv4 payload.

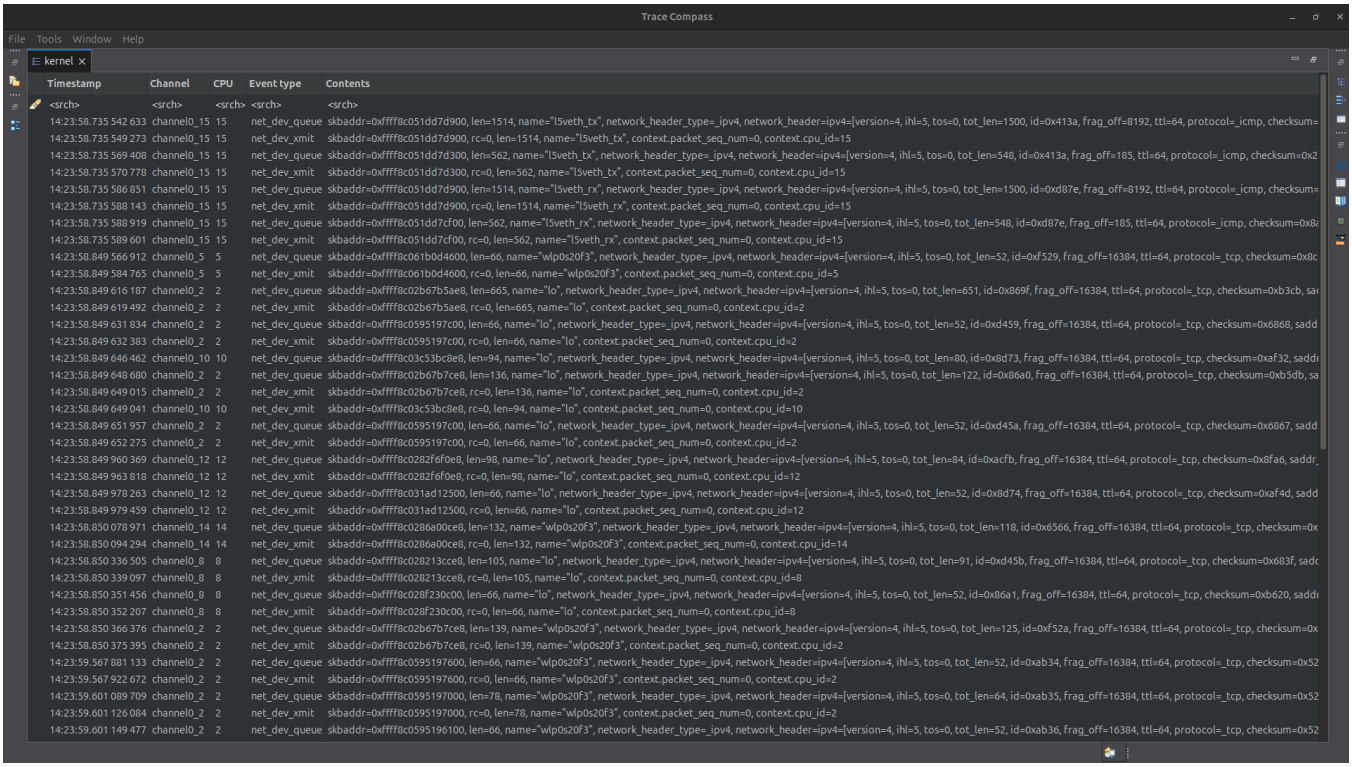


Figure 2 — The event list shows two outgoing request fragments with IPv4 ID `0x413A`.

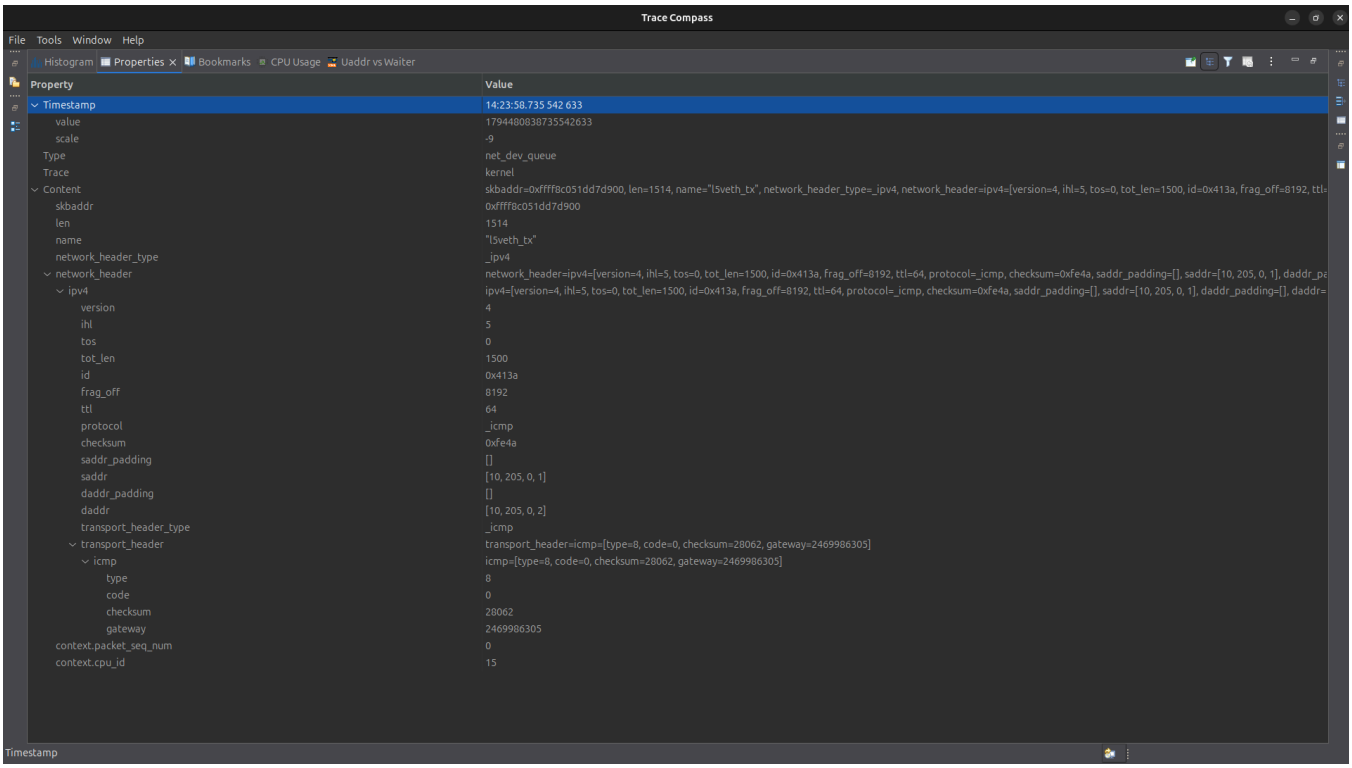


Figure 3 — The first fragment has `tot_len=1500`, `frag_off=8192`, and MF set.

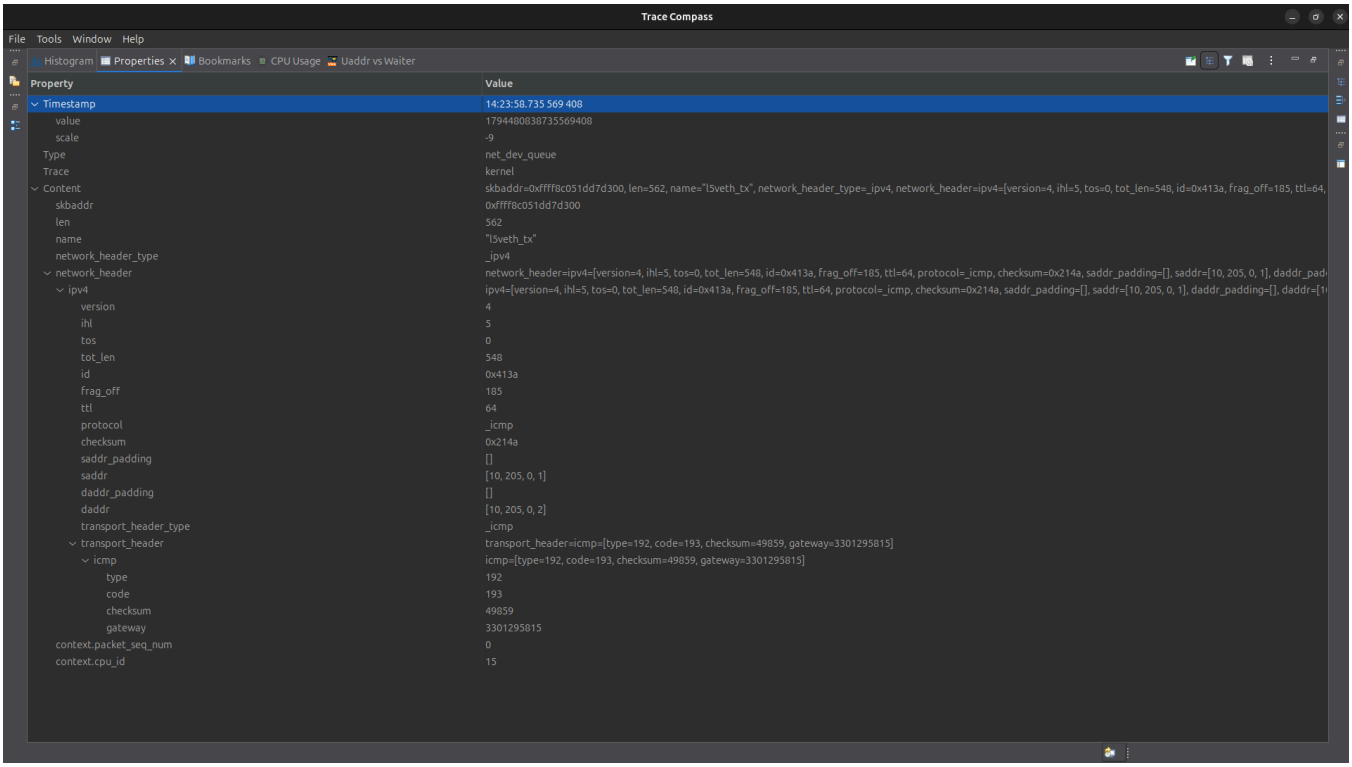


Figure 4 — The final fragment has `tot_len=548` and offset `185 × 8 = 1480` bytes.

The ICMP header resides in the **first** fragment. The second fragment contains the continuation of the original datagram's data. Its automatically displayed ICMP-like fields are a decoding artifact; I interpreted that fragment using its IPv4 length, identification, and offset.

4.3 Deriving the original message length from Trace Compass

I used the `tot_len` values and the header length to recover the size of the original IPv4 datagram:

Quantity	Value
Data in fragment 1	`1500 - 20 = 1480` B
Data in fragment 2	`548 - 20 = 528` B
Reassembled IPv4 payload (ICMP message)	**2008 B**
One original IPv4 header	20 B
Original IPv4 datagram	**2028 B**
ICMP header	8 B
Original user Ping data	**2000 B**

Equivalently:

Original IPv4 size = 1500 + 548 − 20 = 2028 bytes.

Ping data = 2028 − 20 − 8 = 2000 bytes.

Subtracting **one** 20-byte IPv4 header from the sum removes the extra header introduced by fragmentation. The reconstructed **2000-byte Ping payload** matches `ping -s 2000`, demonstrating that the original message length can be recovered from the trace.

5. Additional Packet Capture with tcpdump and Wireshark

After completing the required LTTng analysis, I extended the experiment to inspect the actual bytes of the packets. I installed **Wireshark and TShark 4.2.2** and used `scripts/08_trace_packet_end_to_end.sh` to coordinate a new LTTng recording with `tcpdump` on the transmitter's `l5veth_tx` interface.

I sent **one** 2000-byte Ping request, saved the capture as `logs/pcap/08_ping_2000.pcap`, and inspected it using Wireshark and TShark. The capture contained **four Ethernet frames**, with **zero packets dropped by tcpdump**:

Frame	Direction	IPv4 ID	IPv4 `tot_len`	MF	Offset in bytes
1	Request: 10.205.0.1 → 10.205.0.2	0x9E55	**1500 B**	1	0
2	Request: 10.205.0.1 → 10.205.0.2	0x9E55	**548 B**	0	1480
3	Reply: 10.205.0.2 → 10.205.0.1	0xA982	**1500 B**	1	0
4	Reply: 10.205.0.2 → 10.205.0.1	0xA982	**548 B**	0	1480

Wireshark displayed the first frame of each pair as a fragmented IPv4 packet and identified the final frame as the point at which reassembly completed. The request and reply used **different IPv4 IDs**, as expected for two distinct IPv4 datagrams.

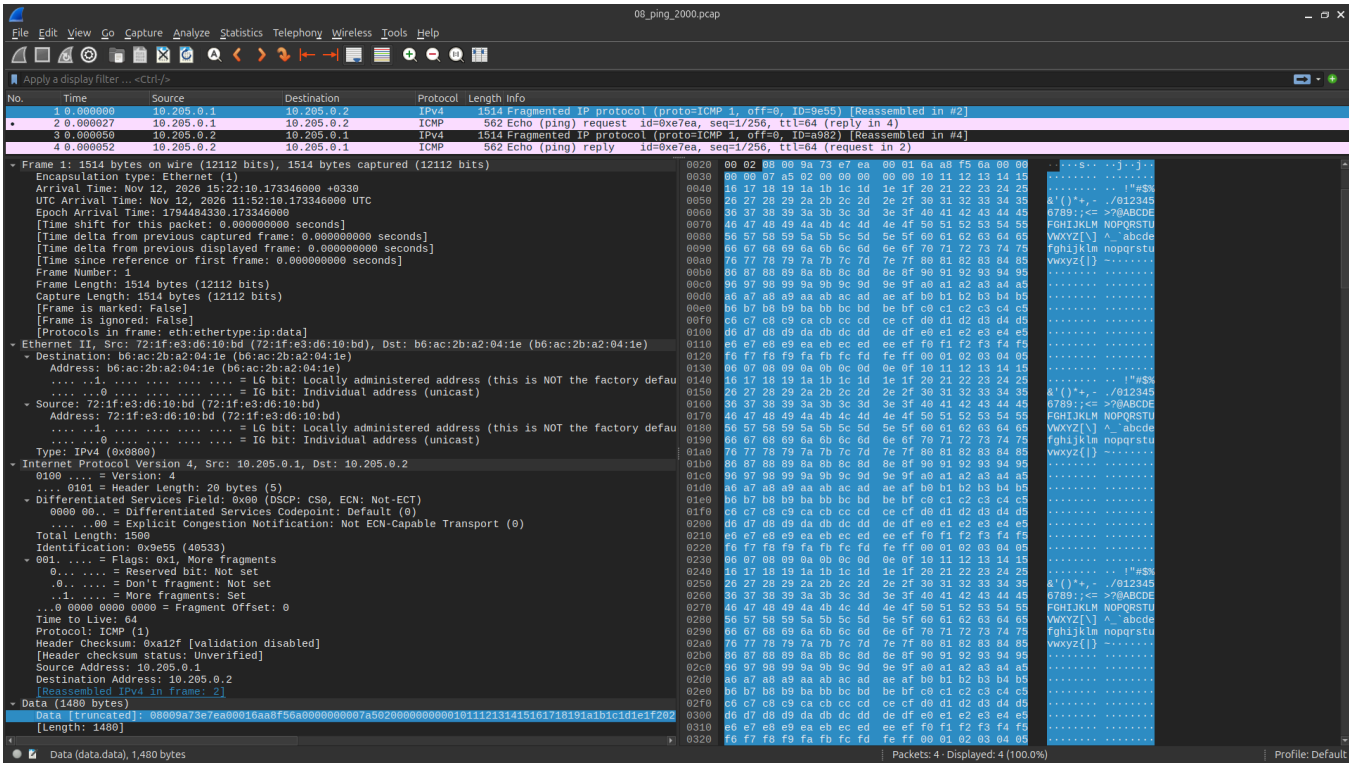


Figure 5 — Wireshark shows four captured frames and the first request fragment's IPv4 fields.

5.1 Reassembly and the original ICMP message

When I selected **frame 2**, Wireshark showed the request's final fragment with `ip.len=548`, offset **1480 bytes**, and MF cleared. It also displayed the reassembly information:

```
[2 IPv4 Fragments (2008 bytes): #1(1480), #2(528)]
```

The reassembled **2008 bytes** represent the complete ICMP message: an **8-byte ICMP header plus 2000 bytes of Ping data**. With the original IPv4 header, this is the same **2028-byte** datagram inferred from the LTTng events. Wireshark was then able to interpret the ICMP Echo Request (type 8, code 0) and its checksum.

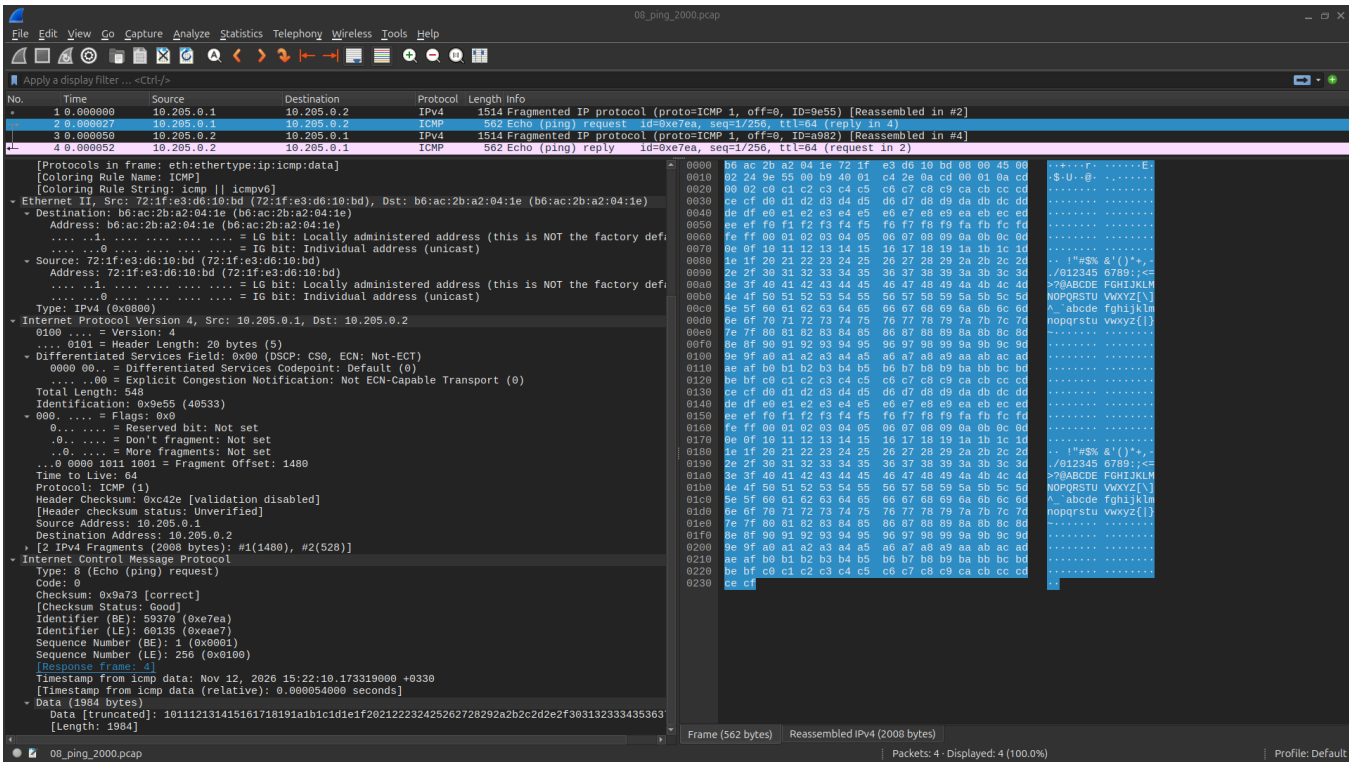


Figure 6 — The final request fragment completes the 2008-byte ICMP message.

The packet-bytes pane showed the captured headers and payload. Wireshark reconstructed the ICMP message from the PCAP, while the successful Ping reply confirmed the endpoint exchange. The 2008-byte reassembly includes the ICMP header and 2000 bytes of Ping data. Within that data, Wireshark decodes a 16-byte timestamp separately and displays the remaining **1984 bytes** as Data.

TShark's exported `ip.frag_offset` value is **185** in eight-byte units; Wireshark displays the corresponding **1480-byte** offset.

6. Relating the Application's System Calls to Fragment Transmission

During the coordinated capture, I enabled LTTng system-call tracing for `socket`, `setsockopt`, `sendto`, `sendmsg`, `recvfrom`, and `recvmsg`, as well as network and scheduling events. I added `vpid`, `vtid`, and `procname` contexts so that I could distinguish the Ping process from unrelated system activity.

The session generated **14,234 system-wide events** in `logs/lttng/08_packet_events.txt`, including **5596 `sched_switch` events**. I used process context and interface names to identify the experiment's `net_dev_queue` and `net_dev_xmit` events.

6.1 Identifying the Ping process and socket

The relevant application had `vpid=124906`, `vtid=124906`, and `procname="ping"`. Filtering only the PID also displayed earlier namespace setup under `procname="ip"`; I used the process name to distinguish the Ping socket calls.

The Ping trace showed `socket(family=2, type=2, protocol=1)` returning `-13`, followed by `socket(family=2, type=3, protocol=1)` returning **3**. The successful call created an IPv4 raw ICMP socket (`AF_INET`, `SOCK_RAW`, `IPPROTO_ICMP`).

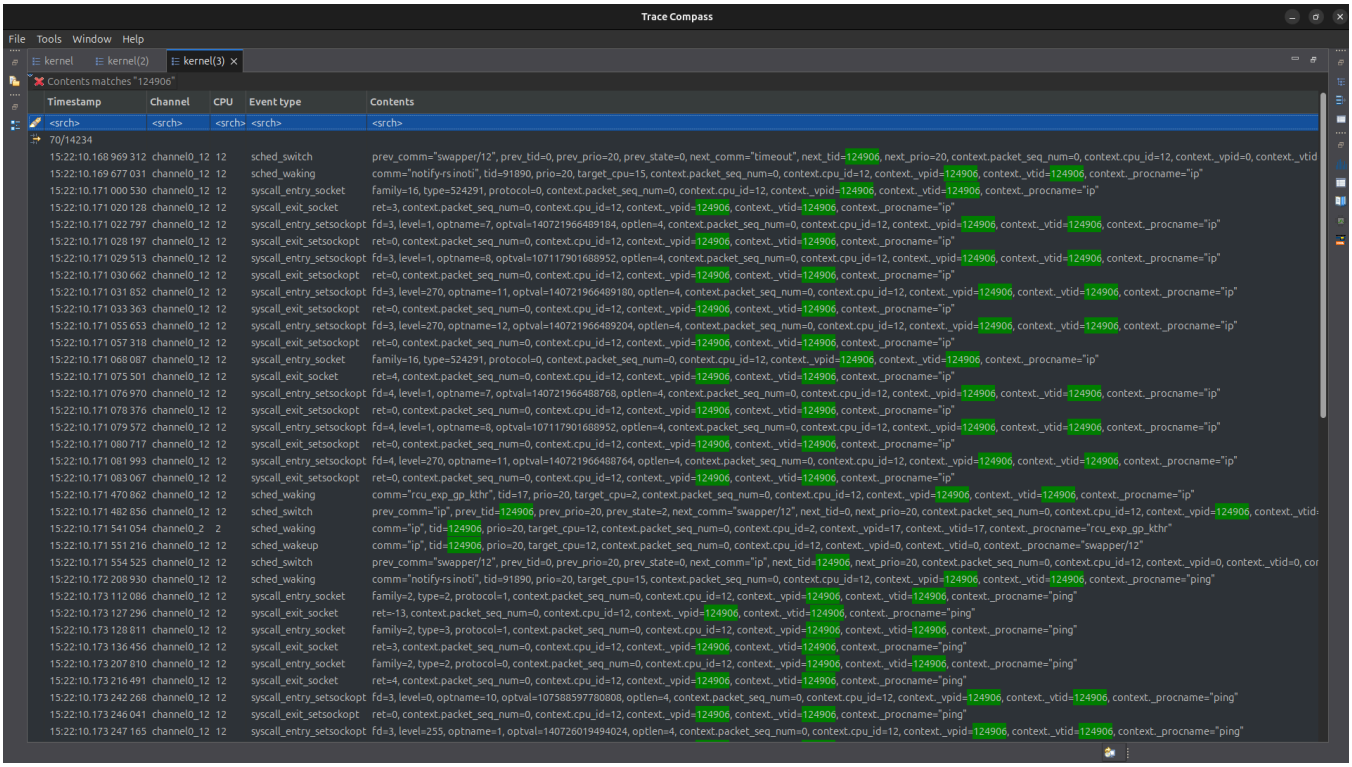


Figure 7 — Filtering PID 124906 shows namespace setup under `ip`, followed by the sockets opened by `ping`.

6.2 The observed send-to-fragment timeline

The following times came from the **coordinated LTTng recording**:

Timestamp (15:22:10)	Event	Evidence
.173320854	`syscall_entry_sendto`	`ping`, PID/TID 124906, `fd=3`, **`len=2008`**
.173342520	`net_dev_queue`	`l5veth_tx`, first fragment: `len=1514`, `tot_len=1500`, ID `0x9E55`
.173354483	`net_dev_xmit`	First fragment, `len=1514`, `rc=0`
.173372770	`net_dev_queue`	`l5veth_tx`, second fragment: `len=562`, `tot_len=548`, ID `0x9E55`
.173375500	`net_dev_xmit`	Second fragment, `len=562`, `rc=0`
.173470456	`syscall_exit_sendto`	**`ret=2008`**
.173497864	`syscall_entry_recvmsg`	`ping` starts reading a received message
.173505264	`syscall_exit_recvmsg`	**`ret=2028`**

Both transmit fragment events occurred under the Ping process context (`vpid=124906` and `procname="ping"`) and had the same IPv4 ID, **`0x9E55`**, as the request fragments in Wireshark. This gave me a direct connection between the application system call, the kernel network events, and the captured packet bytes.

The first `net_dev_queue` event occurred approximately **21.666 microseconds** after `syscall_entry_sendto`, and the second approximately **51.916 microseconds** after that entry. These intervals describe the observed progression from syscall entry to the two transmit-queue events in this recording.

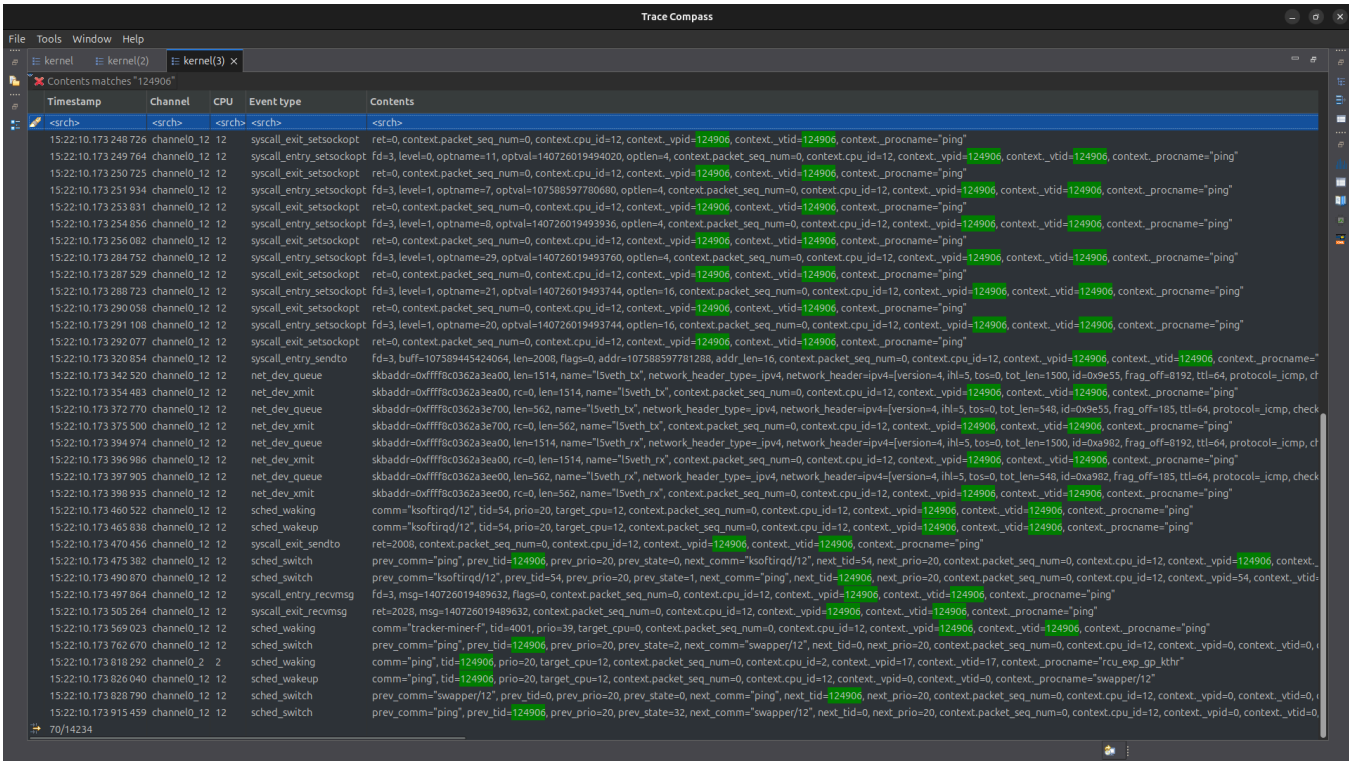


Figure 8 — The `sendto`, fragment transmission, and `recvmsg` events appear in order for PID 124906.

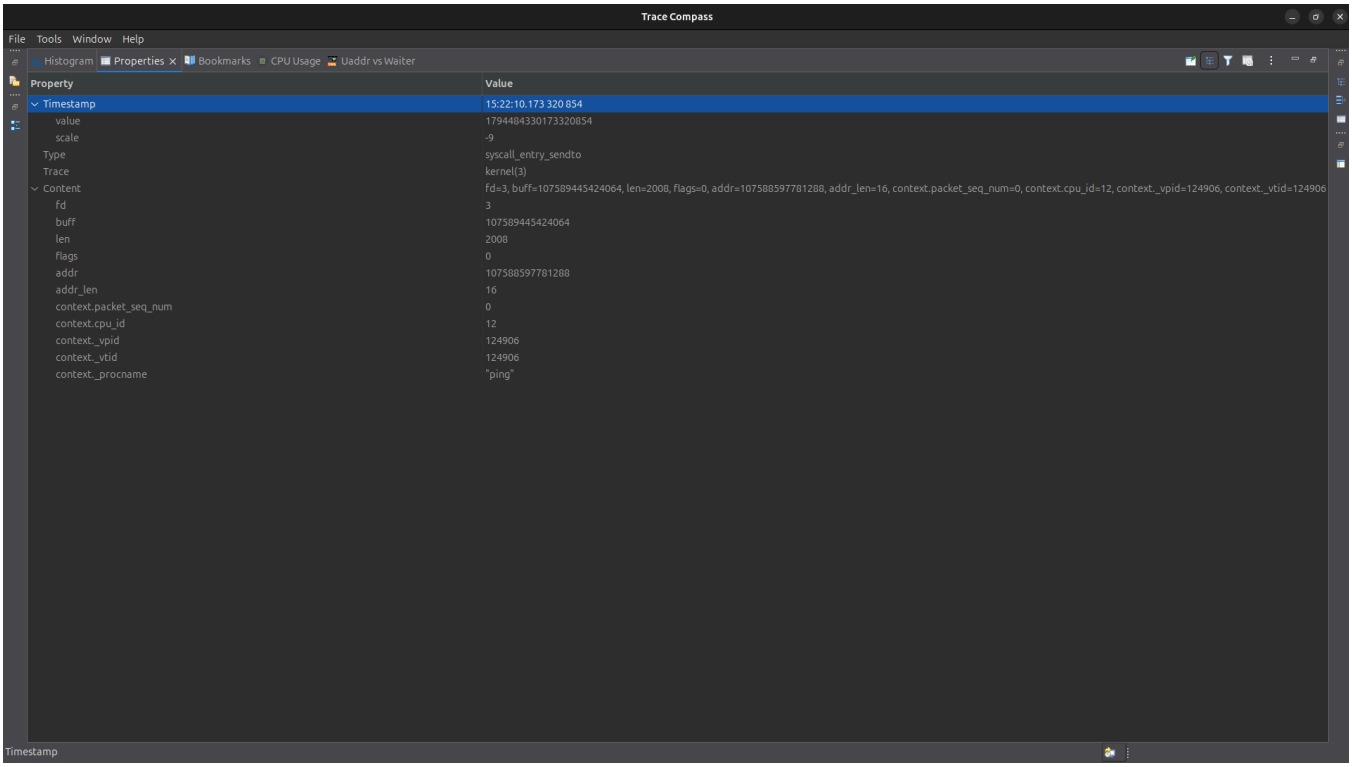


Figure 9 — The `sendto` entry records file descriptor 3 and a 2008-byte message from `ping`.

The `sendto` return of **2008** records acceptance of the complete ICMP message. On this raw IPv4 socket, the received message includes the IPv4 header, which accounts for `recvmsg` returning **2028 = 2008 + 20 bytes**. The Ping reply and captured reply fragments confirmed the exchange.

7. Additional ftrace Function Graph of IPv4 Fragmentation

To examine kernel functions between packet preparation and device transmission, I performed a **separate additional experiment** using ftrace's `function_graph` tracer. I first checked available symbols with `scripts/09_ftrace_fragmentation_preflight.sh`. The relevant functions included `ip_fragment.constprop.0`, `ip_do_fragment`, `ip_fraglist_init`, `ip_fraglist_prepare`, `ip_frag_next`, `ip_finish_output2`, and `__dev_queue_xmit`.

I then used `scripts/10_ftrace_ip_fragment.sh` to set `ip_fragment.constprop.0` as the graph root and send one oversized Ping through the same namespace network. The saved graph contained **1398 lines**, approximately **106 KB**, at `logs/ftrace/10_ip_fragment.graph`. The Ping succeeded. After saving the graph, the script ran its cleanup routine to restore the previous tracer settings.

7.1 Actual kernel function path

The recorded function graph contained the following nested sequence (shortened to the functions relevant to the assignment):

```
ip_fragment.constprop.0() {
    ip_do_fragment() {
        ip_fraglist_init();
        ip_fraglist_prepare();

        ip_finish_output2() {
            __dev_queue_xmit() {
                dev_hard_start_xmit() {
                    veth_xmit();
                }
            }
        }

        ip_finish_output2() {
            __dev_queue_xmit() {
                dev_hard_start_xmit() {
                    veth_xmit();
                }
            }
        }
    }
}
```

This abbreviated sequence retains the relevant nesting from the saved function graph. The two `ip_finish_output2` branches under one `ip_do_fragment` invocation were consistent with dispatching two fragments through the virtual Ethernet interface. The full file also contained neighbor processing, queueing, and other kernel functions, including unrelated interrupts that occurred on the traced CPU.

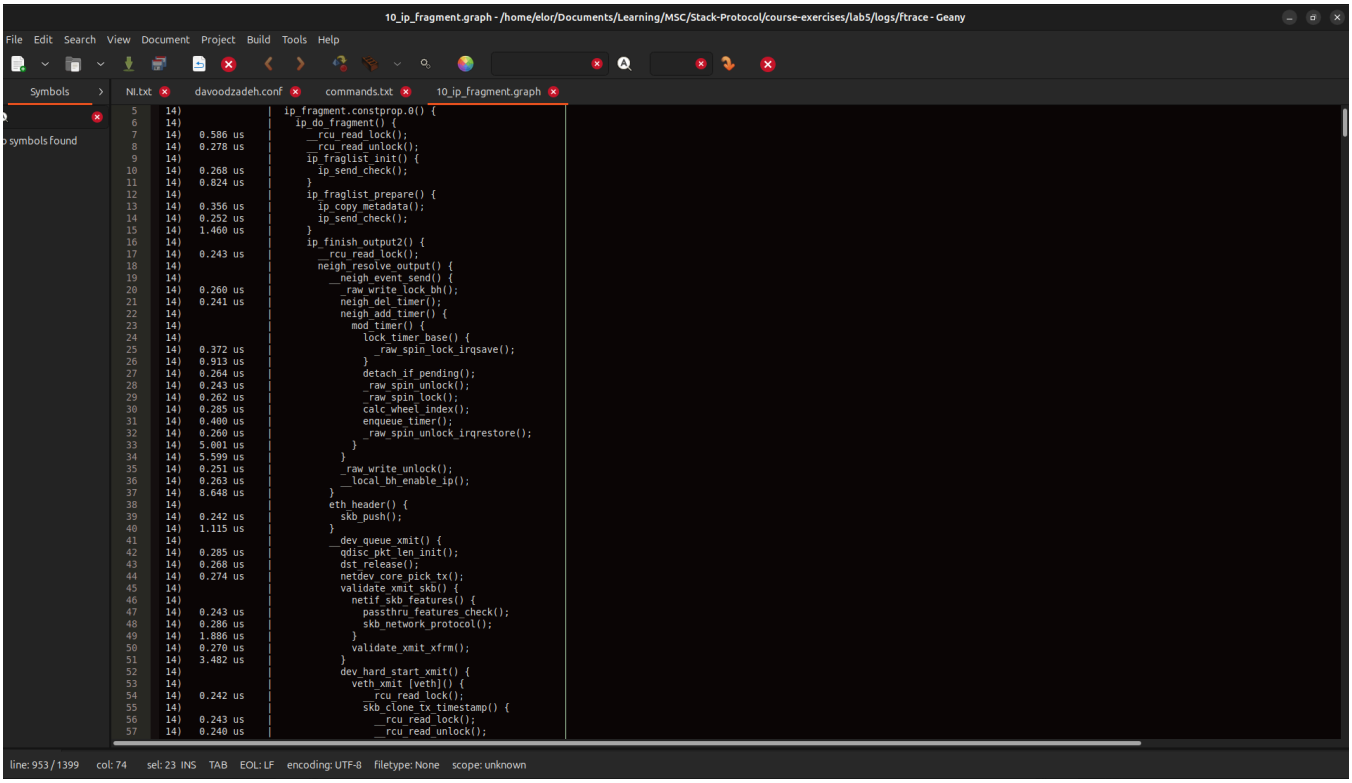


Figure 10 — The first fragmentation path reaches `ip_finish_output2`, `__dev_queue_xmit`, and `veth_xmit`.

7.2 Function occurrence counts

I used `scripts/11_inspect_fragmentation_graph.sh` to count the relevant function calls in the saved graph:

Function	Observed entries
`ip_fragment.constprop.0`	2
`ip_do_fragment`	2
`ip_fraglist_init`	2
`ip_fraglist_prepare`	2
`ip_frag_next`	0
`ip_finish_output2`	4
`__dev_queue_xmit`	4
`dev_hard_start_xmit`	4
`veth_xmit`	4

These counts were consistent with **two fragmented IPv4 datagrams** (one Echo Request and one Echo Reply), with **two outgoing fragments each**. The graph also showed `icmp_echo` and `icmp_reply` before the second fragmentation invocation, connecting that invocation to reply processing in this separate ftrace run.

The calls to `ip_fraglist_init` and `ip_fraglist_prepare` identified the **fraglist-related path** used in this run.

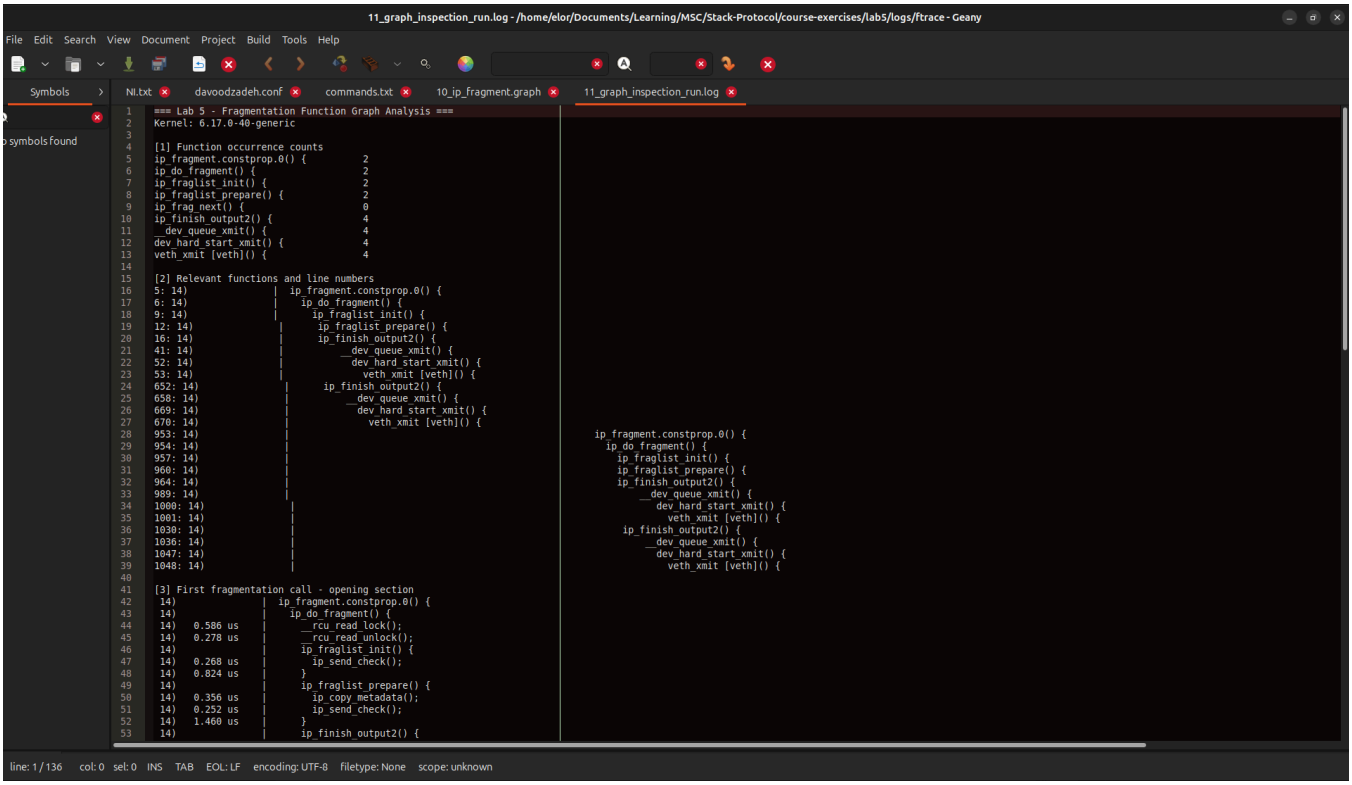


Figure 11 — The saved graph contains two fragmentation entries and four virtual-interface transmit entries.

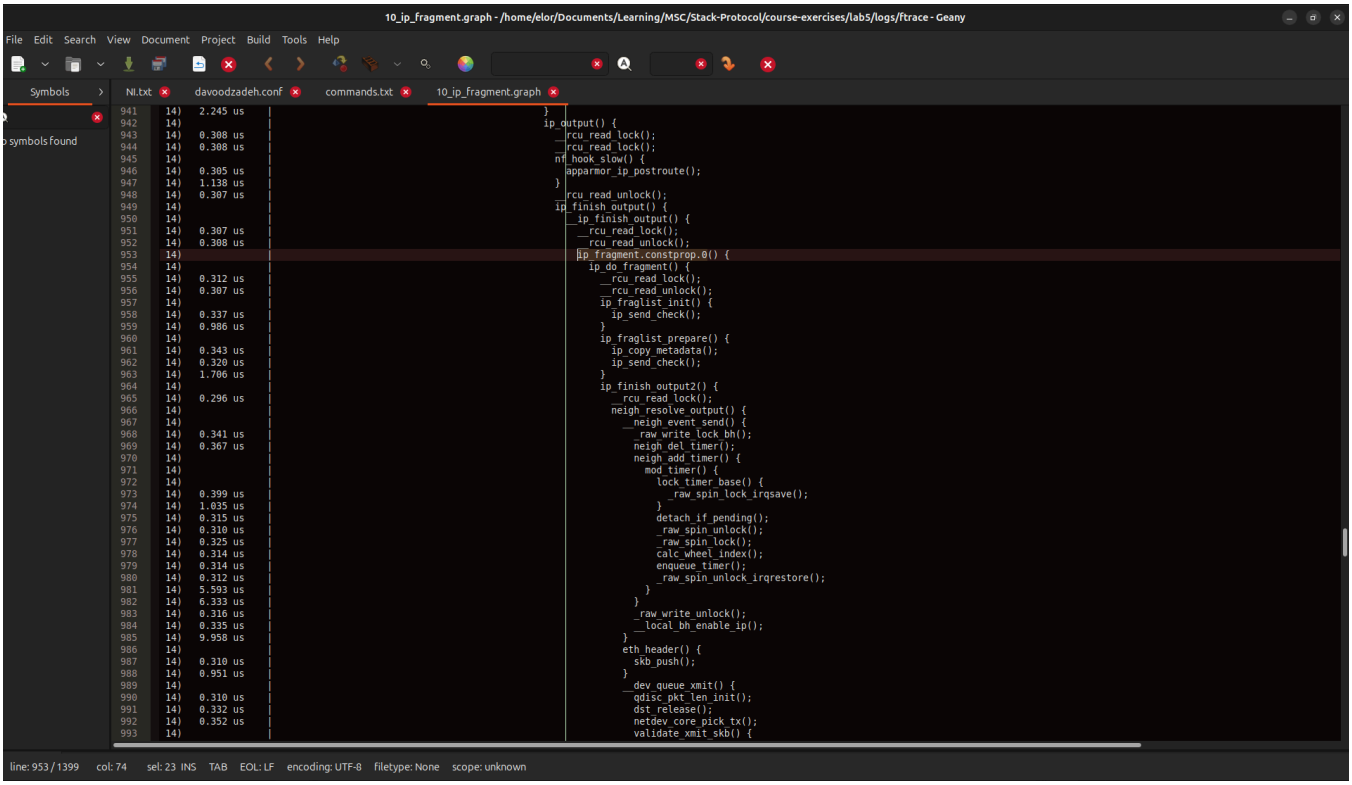


Figure 12 — The second fragmentation invocation appears within the IPv4 output path.